

Informe Taller Angular

Observables y asincronía

- a) Debido a que una petición HTTP es asincrónica; al momento de llamar al servicio, este no retorna una respuesta de forma inmediata. Angular usa *Observable* ya que representa un flujo de datos asincrónico en el cual se pueden ejecutar las operaciones de suscribir, cancelar y encadenar, así como la emisión de más de un valor. Mientras que con *Promise* solamente hay representación para un único valor (futuro), es decir, no es posible reintentar ni cancelar la petición.
- b) No, cada *Observable* devuelto por las peticiones HTTP hace que las peticiones no se ejecuten hasta que alguien se suscribe. En caso que el componente llame al servicio pero no se utilice el método `.subscribe()`, la solicitud HTTP no se dispara y por consiguiente, tampoco llega al backend.

Inyección de dependencias

- c) La utilización de `providedIn: 'root'` permite que Angular cree y comparta una sola instancia del servicio mediante el contenedor de inyecciones de dependencias, mejorando la modularidad y evitando el acoplamiento a la forma exacta presente en el constructor del servicio. Asimismo, facilita la reutilización del mismo servicio en distintos componentes, lo que impide repetir de manera innecesaria la lógica de la creación, configuración o uso de dependencias internas
- d) Debido a que en las pruebas se puede reemplazar fácilmente la implementación real por un "doble"; de esta forma, el componente o servicio se prueba de manera aislada, sin depender de llamadas reales al backend ni a la API, permitiendo mayor velocidad, determinismo y simplicidad constante

Interceptores

- e) Porque el manejo de errores HTTP es un elemento transversal para el proyecto; centralizarlo en el interceptor impide que se presente una repetición innecesaria de la lógica en cada componente, garantizando un comportamiento consistente para la totalidad de la aplicación.
- f) Algunos de los usos comunes adicionales para un interceptor son: Agregación de tokens de autenticación para todas las peticiones y registro / medición de tráfico en HTTP para escenarios de inicio de sesión, métricas y obtención de tiempos de rta

Patrón Maestro - Detalle

- g) Debido a que en el patrón maestro-detalle ya existe una ciudad seleccionada en el componente maestro y la implementación de la etiqueta `@Input` evita que se mande una petición adicional innecesaria. Lo anterior trae consigo una reducción en el tráfico de red, mejoras en el rendimiento y una simplificación en el flujo de datos.
- h) La ventaja se presenta en el frontend, ya que este recibe una estructura más útil y lista para su presentación sin necesidad de realizar consultas adicionales para retornar el país. Esto muestra directamente el nombre del país y la tabla de ciudades, de forma que sea posible trabajar con datos ya relacionados; la implementación presente en el proyecto por medio del método GET simplifica en gran medida la vista del maestro

Link Github: https://github.com/Dazix57/ISIS2603_202610_TAngular2_ndazaa.git